

Technische Internetbasierte Systeme

Speicher, Sleep Modes und Scheduling

Linus Hoppe

Aalen University

May 12, 2026



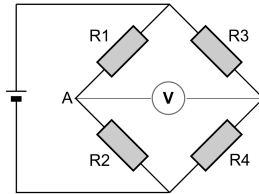
Hochschule Aalen



- ① Rückblick
- ② Superloop
- ③ `yield()`, `delay()` und Watchdog
- ④ Kooperatives Scheduling
- ⑤ RTOS
- ⑥ Dual-Core und echte parallele Aufgaben
- ⑦ Datenblatt-Aufgabe
- ⑧ EEPROM
- ⑨ Sleep Modes

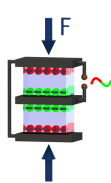


- ➊ Rückblick
- ➋ Superloop
- ➌ `yield()`, `delay()` und Watchdog
- ➍ Kooperatives Scheduling
- ➎ RTOS
- ➏ Dual-Core und echte parallele Aufgaben
- ➐ Datenblatt-Aufgabe
- ➑ EEPROM
- ➒ Sleep Modes



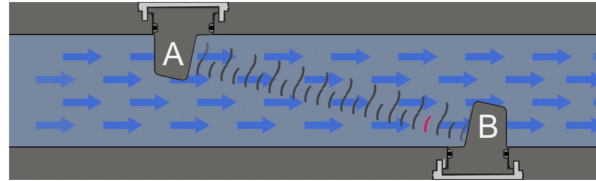
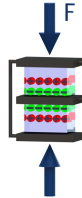
DMS

Kraft $\rightarrow$ Widerstand



Piezo

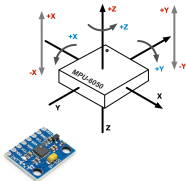
Kraftänderung $\rightarrow$
Spannung



Durchfluss

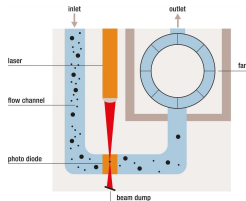
Laufzeitdifferenz

Messen heißt: physikalische Größe $\rightarrow$ elektrisches Signal



IMU

Beschleunigung + Rotation



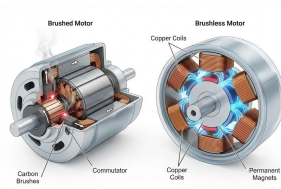
Feinstaub

Lichtstreuung



Wearable

optisch / elektrisch



Aktor

Signal → Bewegung

Sensoren erfassen Zustände. Aktoren verändern Zustände.



- ① Rückblick
- ② Superloop
- ③ `yield()`, `delay()` und Watchdog
- ④ Kooperatives Scheduling
- ⑤ RTOS
- ⑥ Dual-Core und echte parallele Aufgaben
- ⑦ Datenblatt-Aufgabe
- ⑧ EEPROM
- ⑨ Sleep Modes



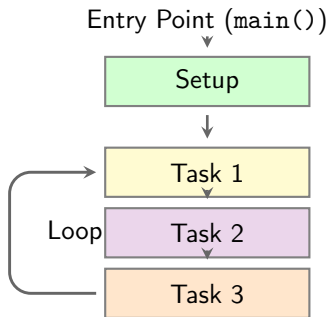
- Einfache Endlosschleife: `loop()`
- Aufgaben werden nacheinander ausgeführt
- Keine echte Parallelität

Eigenschaften

- Kooperatives Verhalten
- Entwickler steuert Ablauf vollständig
- Blockierende Funktionen sind kritisch



Super Loop



- Ablauf: Setup → Task 1 → Task 2 → Task 3
- Danach Rücksprung zu Task 1
- Die Aufgaben werden nacheinander in jeder Runde erneut bearbeitet



Board / Plattform	Typisches Modell	Bemerkung
Arduino Uno / Nano	<code>setup()</code> + <code>loop()</code>	klassisches Arduino-Modell
Arduino Mega	<code>setup()</code> + <code>loop()</code>	gleiche Struktur, mehr I/O
Arduino Leonardo / Micro	<code>setup()</code> + <code>loop()</code>	Arduino-Core auf ATmega32u4
ESP8266	<code>setup()</code> + <code>loop()</code>	Arduino-Core, Hintergrundaufgaben
ATtiny85 / Digispark	<code>setup()</code> + <code>loop()</code>	kleine AVR-Systeme mit Arduino-Core
STM32 Arduino-Core	<code>setup()</code> + <code>loop()</code>	Arduino-Abstraktion auf STM32

Kernaussage

Viele Mikrocontroller-Frameworks nutzen eine einfache Endlosschleife als Grundmodell. Der Superloop ist deshalb ein verbreitetes Muster für kleine eingebettete Systeme.



```
void setup() {  
    pinMode(LED_BUILTIN, OUTPUT);  
}  
  
void loop() {  
    digitalWrite(LED_BUILTIN, HIGH);  
    delay(1000);  
  
    digitalWrite(LED_BUILTIN, LOW);  
    delay(1000);  
}
```

- `delay()` blockiert den Programmablauf
- Währenddessen keine weiteren Aufgaben möglich
- Schlechte Grundlage für mehrere gleichzeitige Aufgaben



- Programme laufen in einer Endlosschleife `loop()`
- Keine echten parallelen Threads
- Mehrere Aufgaben müssen innerhalb einer Schleife organisiert werden
- Problem: Wie mehrere Aufgaben gleichzeitig bearbeiten?
- Lösung: Zeitbasierte Steuerung statt blockierender Aufrufe



```
const int ledPin = LED_BUILTIN;

unsigned long lastToggle = 0;
const unsigned long interval = 1000;

bool ledState = false;

void setup() {
    pinMode(ledPin, OUTPUT);
    lastToggle = millis();
}

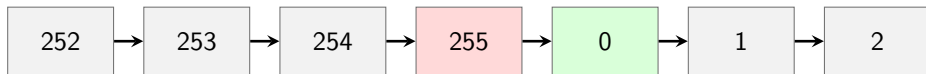
void loop() {
    unsigned long now = millis();

    if (now >= lastToggle + interval) {
        lastToggle = now;
        ledState = !ledState;
        digitalWrite(ledPin, ledState);
    }
}
```

- Keine Blockierung durch `delay()`
- Zeit wird über einen Zielzeitpunkt geprüft
- `loop()` läuft weiter
- Mehrere Aufgaben können ergänzt werden

`millis()`

- Rückgabewert: unsigned long
- Einheit: Millisekunden (ms)
- Start: seit Programmstart



Beispiel mit `uint8_t`: nach 255 folgt wieder 0

Grundidee

- `millis()` ist ein Zähler
- der Zähler hat einen festen Wertebereich
- nach dem Maximalwert folgt wieder 0

Wichtig

Ein Überlauf ist kein Fehler des Prozessors.

Er ist normales Verhalten bei Ganzzahlen mit fester Bitbreite.



```
void setup() {  
    Serial.begin(115200);  
}  
  
void loop() {  
    // Force millis() into 8 bit  
    uint8_t now = millis();  
  
    Serial.println(now);  
    delay(20);  
}
```

Beobachtung

- Werte laufen von 0 bis 255
- danach beginnt der Zähler wieder bei 0
- $\text{now} > 255$ kann nie wahr werden

Kernaussage

Ein Datentyp kann nur Werte innerhalb seines Wertebereichs speichern.



Datentyp	Größe	Wertebereich	Verwendung
uint8_t	8 Bit	0 bis 255	kleines Beispiel
uint16_t	16 Bit	0 bis 65 535	kurze Zeitfenster
uint32_t	32 Bit	0 bis 4 294 967 295	Zeitwerte
unsigned long	32 Bit	0 bis 4 294 967 295	millis()

Überlauf bei millis()

millis() zählt in Millisekunden.

4 294 967 295 ms $\approx$ 49,7 Tage

Danach folgt wieder 0.

Problem: absoluter Zielzeitpunkt



```
unsigned long lastTime = 4294966000UL;
const unsigned long interval = 500;

void loop() {
    unsigned long now = millis();

    unsigned long targetTime = lastTime +
        interval;

    // Problem after overflow
    if (now >= targetTime) {
        lastTime = now;
        // task
    }
}
```

Situation

lastTime = 4 294 966 000

interval = 500

targetTime = 4 294 966 500

Kein Überlauf bei der Zielzeit.

Problem

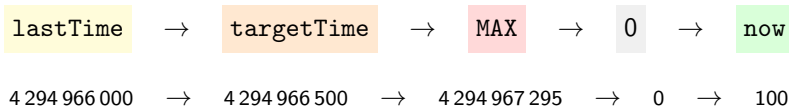
Wenn now danach überläuft, ist now wieder klein.

Dann gilt fälschlich:

Warum der Vergleich falsch wird



Reale Reihenfolge im Zähler



Was real passiert

- targetTime wird noch vor dem Überlauf erreicht
- danach läuft der Zähler über
- now ist dann wieder klein

Was der Vergleich prüft

`now >= targetTime`

`100 ≥ 4 294 966 500`

`false`

Fehler

Der Zeitpunkt ist real schon erreicht, aber der absolute Zahlenvergleich erkennt das nach dem Überlauf nicht mehr.



```
unsigned long lastTime = 4294966000UL;
const unsigned long interval = 500;

void loop() {
    unsigned long now = millis();

    // Rollover-safe time check
    if (now - lastTime >= interval) {
        lastTime = now;
        // task
    }
}
```

Prinzip

Nicht Zielzeit vergleichen:

$$now \geq lastTime + interval$$

sondern vergangene Zeit vergleichen:

$$now - lastTime \geq interval$$

Warum

Unsigned-Arithmetik rechnet zyklisch.
Die Differenz zählt den Abstand über
den Überlauf hinweg.

Warum die Differenz funktioniert



Vereinfachtes Beispiel: Zähler von 0 bis 99

95 $\rightarrow$ 96 $\rightarrow$ 97 $\rightarrow$ 98 $\rightarrow$ 99 $\rightarrow$ 0

Falsch: Zielwert vergleichen

lastTime = 95

interval = 3

targetTime = 98

now = 0

$0 \geq 98 \Rightarrow \text{false}$

Richtig: Abstand vergleichen

lastTime = 95

now = 0

95 $\rightarrow$ 96 $\rightarrow$ 97 $\rightarrow$ 98 $\rightarrow$ 99 $\rightarrow$ 0

5 Schritte vergangen

$5 \geq 3 \Rightarrow \text{true}$

Aufgabe: Mini-Scheduler ohne `delay()`



Zeit: 10 Minuten

Entwickle ein Programm mit mehreren unabhängigen, zeitgesteuerten Aufgaben.

- Built-in-LED periodisch umschalten
- serielle Statusausgabe periodisch senden
- zweiten periodischen Task ergänzen, z. B. Zähler, Sensorwert oder simulierten Messwert
- keine blockierenden Wartefunktionen verwenden
- alle Zeitvergleiche müssen überlaufsicher sein

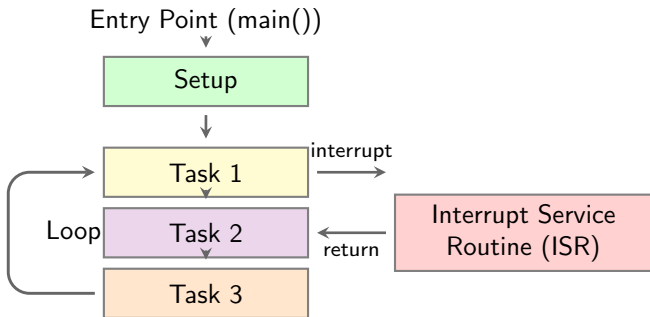
Anforderungen

- LED: 500 ms
- Statusausgabe: 1000 ms
- dritter Task: 3000 ms bis 5000 ms



- 1 Rückblick
- 2 Superloop
- 3 yield(), delay() und Watchdog**
- 4 Kooperatives Scheduling
- 5 RTOS
- 6 Dual-Core und echte parallele Aufgaben
- 7 Datenblatt-Aufgabe
- 8 EEPROM
- 9 Sleep Modes

Super Loop



- Normaler Ablauf: Setup → Task 1 → Task 2 → Task 3
- Danach Rücksprung zu Task 1
- Interrupts können den Ablauf kurz unterbrechen



- Interrupts unterbrechen den normalen Programmablauf
- Die ISR (*Interrupt Service Routine*) sollte sehr kurz sein
- Keine langen Berechnungen in der ISR
- Keine blockierenden Funktionen in der ISR

Typisches Muster

Die ISR setzt nur ein Flag.

Die Verarbeitung erfolgt später in der `loop()`.



- Beim ESP8266 laufen neben dem eigenen Programm auch Systemaufgaben
- Lange blockierende Abschnitte können diese Systemaufgaben verhindern
- yield() gibt explizit Kontrolle an das System zurück
- delay() gibt während der Wartezeit ebenfalls Kontrolle zurück

Einordnung

delay() blockiert die eigene Programmlogik.

yield() blockiert nicht als Wartezeit, sondern erlaubt nur kurz die Ausführung von Hintergrundaufgaben.

Lange Schleife mit yield()



```
void loop() {  
  for (unsigned long i = 0; i < 1000000; i++) {  
    // Long calculation  
  
    if (i % 1000 == 0) {  
      yield(); // Allow system tasks  
    }  
  }  
  
  Serial.println("done");  
}
```

- Schleife läuft weiter
- System bekommt regelmäßig Rechenzeit
- Wichtig bei langen Berechnungen

Regel

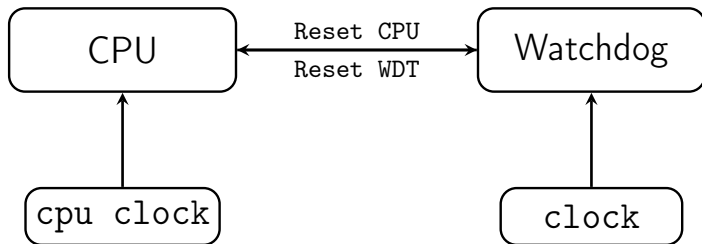
Lange Schleifen regelmäßig unterbrechen.



- Ein Watchdog überwacht, ob das Programm noch reagiert
- Bleibt das Programm zu lange hängen, wird ein Reset ausgelöst
- Dadurch kann sich ein System aus einem blockierten Zustand selbst neu starten

Typische Ursache

- Endlosschleife ohne Rückgabe
- lange Berechnung ohne `yield()`
- blockierende Funktion ohne Zeitbegrenzung



- Die CPU setzt den Watchdog regelmäßig zurück.
- Erfolgt der Rücksetzimpuls nicht innerhalb der vorgegebenen Zeit, löst der Watchdog einen Reset aus.
- Danach löst der Watchdog einen CPU-Reset aus.



```
void loop() {  
  while (true) {  
    // Blocking forever  
    // No yield()  
    // No delay()  
  }  
}
```

- Programm bleibt in der Schleife
- Hintergrundaufgaben laufen nicht mehr
- Watchdog kann Reset auslösen

Problem

Die CPU wird dauerhaft durch eine Aufgabe belegt.



```
----- CUT HERE FOR EXCEPTION DECODER -----  
  
Soft WDT reset  
  
Exception (4):  
epc1=0x402010ef epc2=0x00000000 epc3=0x00000000 excvaddr=0x00000000 depc=0x00000000  
  
>>>stack>>>  
  
ctx: cont  
sp: 3ffffe40 end: 3fffffd0 offset: 0160  
3fffffa0: 011a172c 00000000 3ffee68c 3ffee6b8  
3fffffb0: 00000000 00000000 3ffee68c 40201c7c  
3fffffc0: feefeffe feefeffe 3fffdab0 40100dc1  
<<<stack<<<  
  
----- CUT HERE FOR EXCEPTION DECODER -----
```

Bedeutung

Soft WDT reset bedeutet: Der Software-Watchdog hat einen blockierten Programmablauf erkannt und den ESP8266 zurückgesetzt.



- Werkzeug zum Dekodieren von ESP8266-Exceptions und Stacktraces
- Ordnet Speicheradressen Funktionen und Quellcode-Stellen zu
- Arduino IDE 1.x: <https://github.com/me-no-dev/espexceptiondecoder>
- VS Code: <https://github.com/dankeboy36/esp-exception-decoder>
- Benötigt die passende .elf-Datei der kompilierten Firmware

Wichtig

Die .elf-Datei muss exakt zur geflashten Firmware passen. Nach Codeänderungen muss neu kompiliert werden.

Dekodierte Watchdog-Ausgabe



Raw Crash Output

```
Exception (4):  
epc1=0x402010ef epc2=0x00000000 epc3=0x00000000 excvaddr=0x00000000 depc=0x00000000
```

```
>>>stack>>>
```

```
ctx: cont  
sp: 3ffffe40 end: 3fffffd0 offset: 0160  
3fffffa0: 011a1728 00000000 3fee68c 3fee6b8  
3fffffb0: 00000000 00000000 3fee68c 40201c7c  
3fffffc0: feefeffe feefeffe 3fffdab0 40100dc1  
<<<stack<<<
```

Level1Interrupt

```
Core: 0  
PC: 0x402010ef  
Fault Address: 0x00000000  
Fault Code: 4
```

Stack Trace

#	Address	Function	Location
0	0x40201c7c	loop_wrapper()	core_esp8266_main.cpp
1	0x40100dc1	cont_wrapper	??

Registers

```
EPC1 0x402010ef loop at ??:  
EPC2 0x00000000  
EPC3 0x00000000  
EXCVADDR 0x00000000  
DEPC 0x00000000
```



```
void loop() {  
    while (true) {  
        // Do a small part of the work  
  
        yield(); // Allow system tasks  
    }  
}
```

- System bekommt regelmäßig Kontrolle
- Watchdog wird nicht unnötig ausgelöst
- Trotzdem bleibt die Struktur kritisch

Besser

Lange Aufgaben in kleine Schritte zerlegen und zyklisch ausführen.



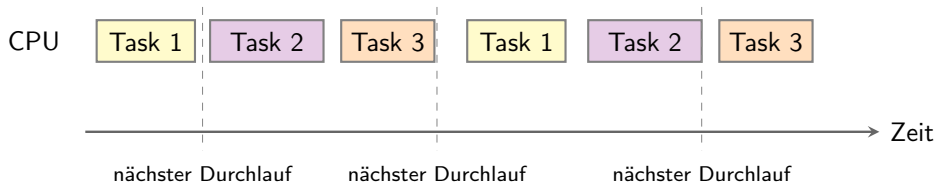
- ① Rückblick
- ② Superloop
- ③ `yield()`, `delay()` und Watchdog
- ④ Kooperatives Scheduling**
- ⑤ RTOS
- ⑥ Dual-Core und echte parallele Aufgaben
- ⑦ Datenblatt-Aufgabe
- ⑧ EEPROM
- ⑨ Sleep Modes



- Mehrere Aufgaben sollen scheinbar gleichzeitig ausgeführt werden
- Im Superloop werden die Aufgaben nacheinander aufgerufen
- Jede Aufgabe darf nur kurz laufen und muss schnell zurückkehren

Prinzip

Nicht eine Aufgabe vollständig abarbeiten,
sondern viele kleine Aufgaben zyklisch aufrufen.



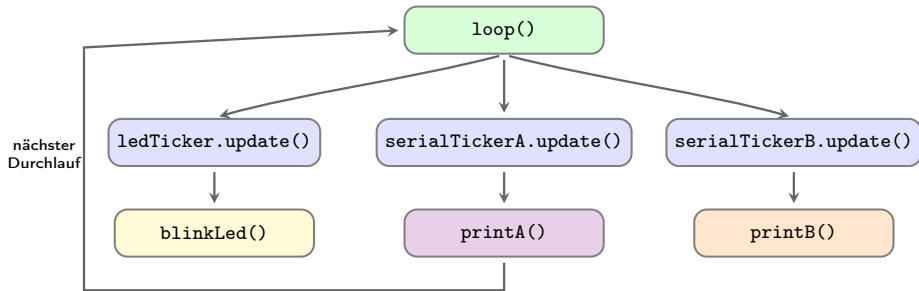
Prinzip: Die Tasks laufen nicht gleichzeitig. Die `loop()` ruft sie regelmäßig auf. Jede Task prüft selbst, ob sie gerade ausgeführt werden muss.

- Eine CPU führt immer nur eine Aufgabe zur gleichen Zeit aus
- Scheduling verteilt die CPU-Zeit auf mehrere Aufgaben
- Kooperativ bedeutet: jede Aufgabe muss schnell zurückgeben



Prinzip

`loop()` prüft regelmäßig die Timer. Ist ein Intervall abgelaufen, wird die passende Callback-Funktion ausgeführt.



- `update()` prüft nur, ob die jeweilige Aufgabe fällig ist
- Callbacks laufen kurz und geben danach wieder zurück



```
#include <TickTwo.h>

const int ledPin = LED_BUILTIN;
bool ledState = false;

void blinkLed() {
    ledState = !ledState;
    digitalWrite(ledPin, ledState);
}

void printA() {
    Serial.println("Ausgabe A");
}

void printB() {
    Serial.println("Ausgabe B");
}

TickTwo ledTicker(blinkLed, 500, 0, MILLIS);
TickTwo serialTickerA(printA, 1000, 0, MILLIS);
TickTwo serialTickerB(printB, 1500, 0, MILLIS);
```

Callback

Eine Callback-Funktion wird nicht direkt in der `loop()` aufgerufen. TickTwo ruft sie auf, wenn ihr Intervall abgelaufen ist.

Beispiel

TickTwo ledTicker(blinkLed, 500, 0, MILLIS);

blinkLed	aufzurufende Funktion
500	Intervall in ms
0	unbegrenzt wiederholen
MILLIS	Zeitbasis Millisekunden

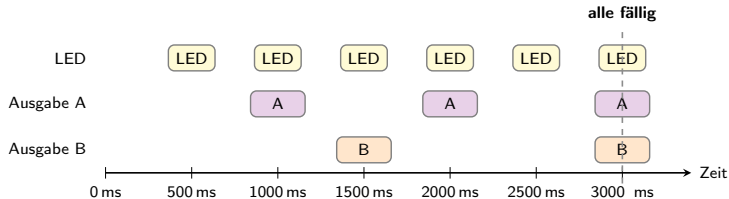
TickTwo: setup(), loop() und Ablauf



Code

```
void setup() {  
  Serial.begin(115200);  
  pinMode(ledPin, OUTPUT);  
  
  ledTicker.start();  
  serialTickerA.start();  
  serialTickerB.start();  
}  
  
void loop() {  
  ledTicker.update();  
  serialTickerA.update();  
  serialTickerB.update();  
}
```

Zeitlicher Ablauf



Frage

Bei 3000 ms sind alle drei Timer fällig.
Welche Callback-Funktion wird zuerst ausgeführt?



```
#include <TickTwo.h>

const int ledPin = LED_BUILTIN;
bool ledState = false;

void blinkLed() {
    ledState = !ledState;
    digitalWrite(ledPin, ledState);
}

void printA() {
    Serial.println("Ausgabe A");
}

void printB() {
    Serial.println("Ausgabe B");
}
```

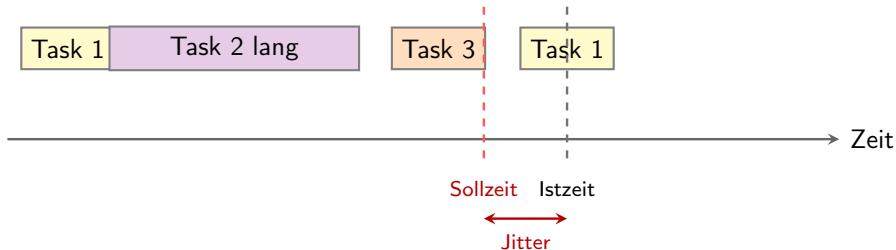
```
TickTwo ledTicker(blinkLed, 500, 0, MILLIS);
TickTwo serialTickerA(printA, 1000, 0, MILLIS);
TickTwo serialTickerB(printB, 1500, 0, MILLIS);

void setup() {
    Serial.begin(115200);
    pinMode(ledPin, OUTPUT);

    ledTicker.start();
    serialTickerA.start();
    serialTickerB.start();
}

void loop() {
    ledTicker.update();
    serialTickerA.update();
    serialTickerB.update();
}
```

Scheduling: Jitter durch lange Tasks



Eine lange Task verzögert den nächsten fälligen Aufruf.

Kernaussage

Tasks im kooperativen Scheduling müssen kurz laufen. Lange Tasks verschieben alle späteren Aufgaben.



- ① Rückblick
- ② Superloop
- ③ `yield()`, `delay()` und Watchdog
- ④ Kooperatives Scheduling
- ⑤ RTOS**
- ⑥ Dual-Core und echte parallele Aufgaben
- ⑦ Datenblatt-Aufgabe
- ⑧ EEPROM
- ⑨ Sleep Modes



- RTOS steht für *Real-Time Operating System*
- Ein RTOS verwaltet mehrere unabhängige Tasks
- Der Scheduler entscheidet, welche Task ausgeführt wird
- Tasks können Prioritäten besitzen
- Zeitvorgaben können systematisch berücksichtigt werden

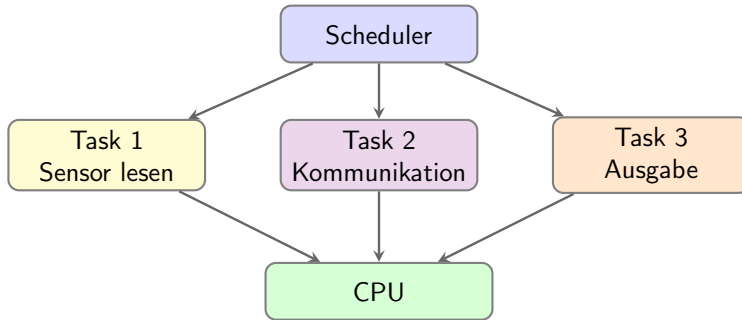
Wichtige Einordnung

Ein RTOS macht ein System nicht automatisch schneller.

Ein RTOS stellt Mechanismen bereit, damit Reaktionen innerhalb definierter Zeitgrenzen organisiert werden können.

Merksatz

Echtzeit bedeutet nicht maximale Geschwindigkeit, sondern vorhersagbares Zeitverhalten.



Der Scheduler verteilt die verfügbare CPU-Zeit auf mehrere Tasks.
Auf einer einzelnen CPU läuft trotzdem immer nur eine Task gleichzeitig.

Vergleich: Superloop vs. RTOS



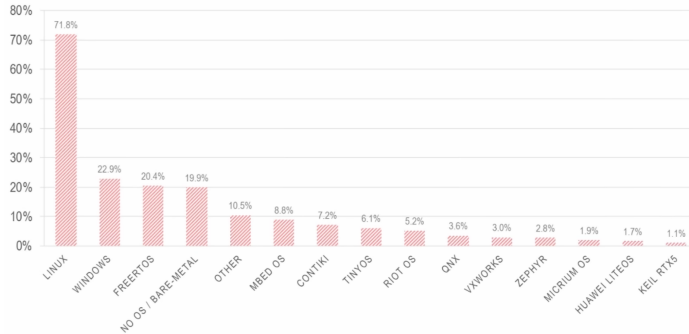
Aspekt	Superloop	RTOS
Grundmodell	Zentrale Endlosschleife	Mehrere Tasks
Ablaufsteuerung	Durch Programmstruktur	Durch Scheduler
Multitasking	Kooperativ	Präemptiv möglich
Blockierung	Betrifft gesamten Ablauf	Betrifft meist nur eine Task
Prioritäten	Manuell nachbildbar	Direkt vorgesehen
Zeitverhalten	Abhängig von Task-Laufzeiten	Systematischer planbar
Komplexität	Gering	Höher

Kernunterschied

Superloop: Aufgaben müssen freiwillig schnell zurückkehren.

RTOS: Der Scheduler verwaltet, unterbricht und aktiviert Tasks.

Which operating system(s) do you use for your IoT devices?



Einordnung

Für kleine Mikrocontroller ist ein RTOS nur sinnvoll, wenn mehrere zeitkritische Aufgaben strukturiert verwaltet werden müssen.



Board / Plattform	RTOS-Status	Typische Einordnung
ESP32 DevKit	integriert	FreeRTOS im System vorhanden
STM32 Nucleo	üblich	FreeRTOS / CMSIS-RTOS
Arduino Portenta H7	möglich	Mbed OS / FreeRTOS
Raspberry Pi Pico / Pico W	möglich	FreeRTOS-Port verfügbar
Teensy 4.1	möglich	RTOS über Portierung
ESP8266	nicht mäßig	standard- Superloop, yield(), Ticker

Hinweis

ESP8266 und ESP32 unterscheiden sich hier deutlich: Beim ESP32 ist FreeRTOS Teil der Plattform, beim ESP8266 im Arduino-Kontext nicht.



```
TaskHandle_t blinkHandle = NULL;

void blinkTask(void *parameter) {
    const int ledPin = 2;
    pinMode(ledPin, OUTPUT);

    while (true) {
        digitalWrite(ledPin, !digitalRead(ledPin));
        // Block only this task for 500 ms
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}

void setup() {
    Serial.begin(115200);

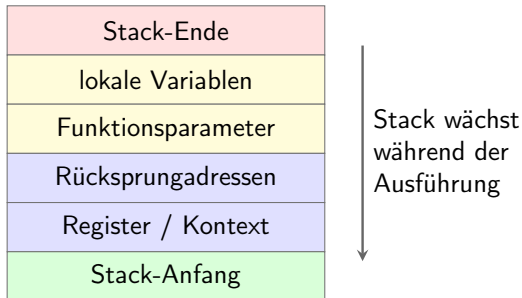
    xTaskCreatePinnedToCore(
        blinkTask,
        "BlinkTask",
        2048,
        NULL,
        1,
        &blinkHandle,
        1
    );
}
```

Parameter

Wert	Bedeutung
blinkTask	Task-Funktion
"BlinkTask"	Name der Task
2048	Stack-Größe
NULL	kein Parameter
1	Priorität
&blinkHandle	Task-Handle
1	Core 1

Wichtig

`vTaskDelay()` blockiert nur diese Task.
Andere Tasks können weiterlaufen.



- Jede Task besitzt ihren eigenen Stack
- Die Stack-Größe wird beim Erzeugen der Task festgelegt
- Der Stack wird bei Funktionsaufrufen und lokalen Variablen genutzt
- Große lokale Arrays erhöhen den Stack-Bedarf
- Zu kleiner Stack kann Abstürze oder undefiniertes Verhalten erzeugen

Bezug zum Code

```
xTaskCreatePinnedToCore(...,  
2048, ...) reserviert den Stack für  
diese Task.
```




- ① Rückblick
- ② Superloop
- ③ `yield()`, `delay()` und Watchdog
- ④ Kooperatives Scheduling
- ⑤ RTOS
- ⑥ Dual-Core und echte parallele Aufgaben**
- ⑦ Datenblatt-Aufgabe
- ⑧ EEPROM
- ⑨ Sleep Modes



- Einige Mikrocontroller besitzen zwei CPU-Kerne
- Aufgaben können dann wirklich parallel laufen
- Beispiel: ESP32 mit Core 0 und Core 1
- Der ESP32-S3 besitzt ebenfalls eine Dual-Core-Architektur



Einordnung

Beim ESP8266 gibt es nur einen Kern.

Dort entsteht Parallelität nur scheinbar durch schnelles Umschalten.



```
void taskCore0(void *parameter) {
    while (true) {
        Serial.println("Task on Core 0");
        delay(1000);
    }
}

void taskCore1(void *parameter) {
    while (true) {
        Serial.println("Task on Core 1");
        delay(700);
    }
}

void setup() {
    Serial.begin(115200);

    xTaskCreatePinnedToCore(
        taskCore0, "TaskCore0", 1000,
        NULL, 1, NULL, 0);

    xTaskCreatePinnedToCore(
        taskCore1, "TaskCore1", 1000,
        NULL, 1, NULL, 1);
}

void loop() {
}
```

- Zwei FreeRTOS-Tasks
- Jeweils fest an einen Core gebunden
- loop() bleibt leer

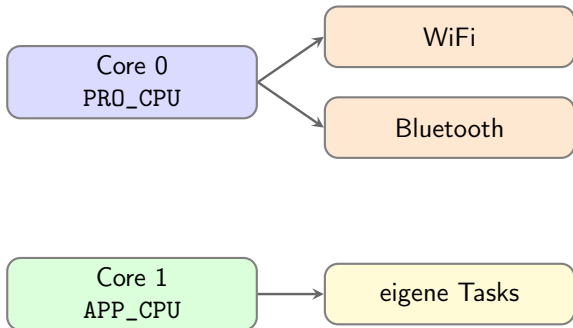
Kernidee

Hier laufen Aufgaben tatsächlich parallel.



Einordnung

- Core 0 wird in ESP-IDF auch PRO_CPU genannt
- Core 1 wird auch APP_CPU genannt
- Protokollnahe Aufgaben wie WiFi und Bluetooth laufen typischerweise auf Core 0
- Anwendungslogik wird häufig auf Core 1 gelegt



Wichtig

Core 0 ist nicht verboten.

Für eigene lange Tasks ist Core 1 meist die robustere Wahl.

Praxisregel

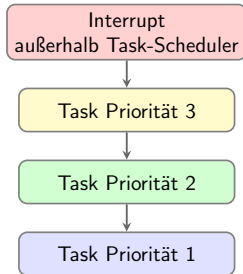
Eigene rechenintensive Tasks nicht unnötig auf Core 0 pinnen.



```
xTaskCreatePinnedToCore(  
    taskCore0,    // function  
    "Task0",      // name  
    1000,         // stack size  
    NULL,         // parameter  
    2,            // priority  
    NULL,         // task handle  
    1);           // core
```

Wichtig

- Größere Zahl = höhere Task-Priorität
- Höchste bereite Task bekommt CPU-Zeit
- Priorität macht Code nicht schneller



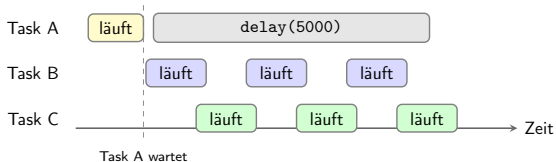
oben wird früher ausgeführt

Achtung

Interrupts können Tasks unabhängig von deren Priorität unterbrechen.



```
void taskA(void *p) {  
    while (true) {  
        delay(5000);  
    }  
}
```



Bedeutung

- `delay()` blockiert nur diese Task
- Die Task gibt CPU-Zeit an den Scheduler ab
- Andere Tasks können weiterlaufen

Einordnung

`delay()` hält nicht die CPU an. Die aktuelle Task schläft, der Scheduler führt andere Tasks aus.



```
volatile int counter = 0;

void task1(void *p) {
    while (true) {
        counter++;
    }
}

void task2(void *p) {
    while (true) {
        Serial.println(counter);
    }
}
```

Frage

Was passiert hier, wenn beide Tasks gleichzeitig auf counter zugreifen?



```
SemaphoreHandle_t mutex;  
volatile int counter = 0;  
  
void task1(void *p) {  
    while (true) {  
        xSemaphoreTake(mutex, portMAX_DELAY);  
  
        counter++;  
  
        xSemaphoreGive(mutex);  
    }  
}
```

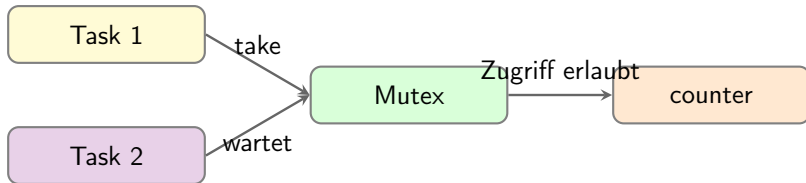
Prinzip

Ein Mutex schützt einen kritischen Abschnitt.

Während eine Task den Mutex besitzt, darf keine andere Task gleichzeitig auf dieselbe Ressource zugreifen.

Ablauf

- 1 Task erreicht `xSemaphoreTake(...)`
- 2 Ist der Mutex frei, bekommt die Task Zugriff
- 3 Die Task führt `counter++` aus
- 4 Mit `xSemaphoreGive(...)` wird der Mutex freigegeben
- 5 Wartende Tasks können danach fortsetzen

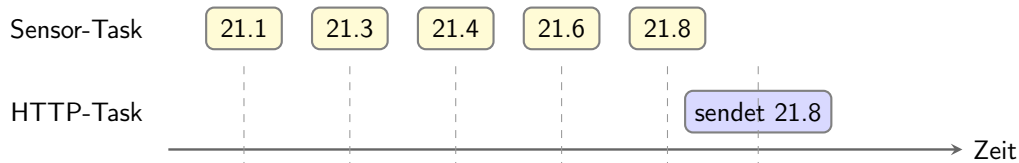


Nur die Task mit Mutex darf den kritischen Abschnitt ausführen.
Nach `xSemaphoreGive()` kann die nächste Task fortsetzen.

Problem: Warum ein Mutex nicht genügt



Beispiel: Sensor-Task und HTTP-Task



Der Sensor erzeugt mehrere Messwerte.
Die HTTP-Task sendet später nur den letzten Wert.
Zwischenwerte gehen verloren.

Einordnung

Ein Mutex schützt die gemeinsame Variable, aber speichert keine Folge von Messwerten.
Für genau dieses Problem braucht man eine **Queue**.

Problem: Mutex reicht hier nicht aus



```
SemaphoreHandle_t mutex;
float latestValue;

void sensorTask(void *p) {
    while (true) {
        float v = readSensor();

        xSemaphoreTake(mutex, portMAX_DELAY);
        latestValue = v;
        xSemaphoreGive(mutex);

        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

void httpTask(void *p) {
    while (true) {
        float v;

        xSemaphoreTake(mutex, portMAX_DELAY);
        v = latestValue;
        xSemaphoreGive(mutex);

        sendToServer(v);
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}
```

Was ist das Problem?

- Sensor liefert z. B. alle 100 ms einen neuen Wert
- HTTP sendet nur alle 1000 ms
- Dazwischen werden viele Werte überschrieben
- Es bleibt nur der *letzte* Messwert erhalten

Wichtig

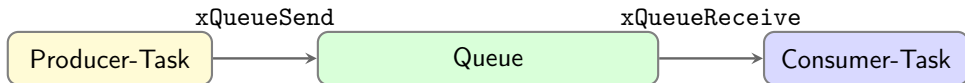
Der Mutex verhindert nur gleichzeitigen Zugriff.

Er verhindert **keinen Datenverlust**.



Prinzip

- Eine Task erzeugt Daten
- Eine andere Task verarbeitet diese Daten
- Die Queue puffert Daten zwischen beiden Tasks
- Erzeuger und Verbraucher müssen nicht gleich schnell arbeiten



Die Queue speichert Nachrichten zwischen.
Die Verarbeitung kann dadurch zeitlich entkoppelt werden.

Wichtig

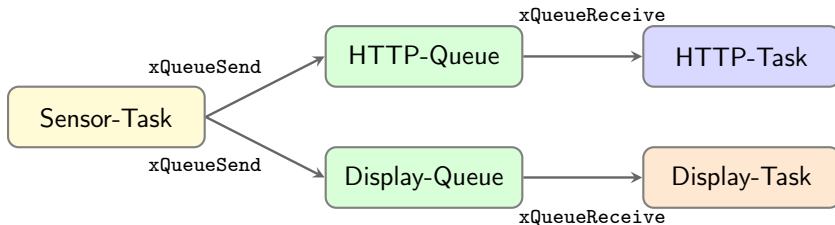
Eine Queue ist ein begrenzter Puffer. Wenn sie voll ist, können neue Daten blockieren oder verloren gehen.



- Sensor-Task misst Werte
- Netzwerk-Task sendet Werte
- Display-Task zeigt Werte an

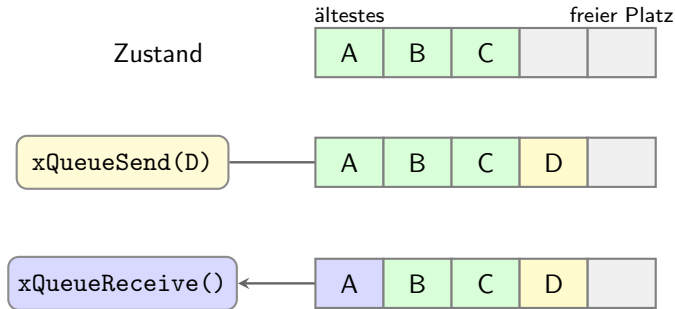
Prinzip

Eine Queue entkoppelt Erzeuger und Verbraucher.



Eine Sensor-Task kann dieselben Messdaten an mehrere Queues senden.
HTTP-Task und Display-Task arbeiten danach unabhängig voneinander.

Queue: Einfügen und Entnehmen



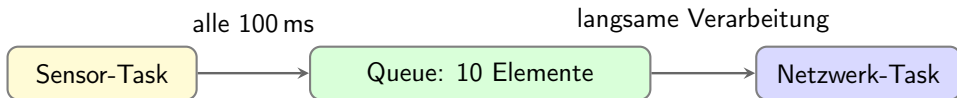
Neues Element wird hinten eingefügt.
Die vorhandenen Elemente bleiben erhalten.

Das älteste Element wird entnommen.
Queues arbeiten nach dem FIFO-Prinzip.

FIFO: First In, First Out.
Das zuerst eingefügte Element wird zuerst wieder entnommen.

Beispiel

- Sensor-Task erzeugt alle 100 ms einen Messwert
- Netzwerk-Task verarbeitet Werte deutlich langsamer
- Die Queue puffert Messwerte zwischendurch
- Ist die Queue voll, gehen neue Werte verloren



Solange noch Platz in der Queue ist, werden Messwerte zwischengespeichert.
Ist die Queue voll, schlägt `xQueueSend(...)` nach maximal 100 ms fehl.

Wichtig

Eine Queue verhindert nicht unbegrenzt Datenverlust. Sie stellt nur einen **begrenzten Puffer** bereit.

Queue-Beispiel in Arduino / FreeRTOS (1)



```
#include <Arduino.h>

QueueHandle_t sensorQueue;

struct SensorValue {
    float temperature;
    uint32_t timestamp;
};

float readTemperature() {
    static float value = 20.0;
    value += 0.1;
    return value;
}

void sendValue(float temperature,
               uint32_t timestamp) {
    Serial.print("Temperature: ");
    Serial.print(temperature);
    Serial.print(" C, time: ");
    Serial.println(timestamp);
}
```

```
void setup() {
    Serial.begin(115200);

    sensorQueue = xQueueCreate(10, sizeof(
        SensorValue));

    if (sensorQueue == NULL) {
        Serial.println("Queue creation failed");
        while (true) {
            delay(1000);
        }
    }

    xTaskCreate(sensorTask, "sensor",
                2048, NULL, 1, NULL);

    xTaskCreate(networkTask, "network",
                4096, NULL, 1, NULL);
}
```


Queue-Beispiel in Arduino / FreeRTOS (2)



```
void sensorTask(void *parameter) {
    SensorValue value;

    while (true) {
        value.temperature = readTemperature();
        value.timestamp = millis();

        if (xQueueSend(
            sensorQueue,
            &value,
            pdMS_TO_TICKS(100)) != pdTRUE) {

            Serial.println(
                "Queue full, value lost");
        }

        vTaskDelay(
            pdMS_TO_TICKS(100)
        );
    }
}
```

```
void networkTask(void *parameter) {
    SensorValue value;

    while (true) {
        if (xQueueReceive(sensorQueue,
            &value,
            portMAX_DELAY)) {

            sendValue(value.temperature, value.timestamp);

            while (xQueueReceive(
                sensorQueue,
                &value,
                0) == pdTRUE) {

                sendValue(value.temperature, value.timestamp);

                // bad code: show -> Queue full, value lost
                vTaskDelay(pdMS_TO_TICKS(500));
            }
        }

        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

void loop() {
}
```



- Queue wird zu klein gewählt
- Rückgabewert von `xQueueSend()` wird ignoriert
- Große Datenstrukturen werden unnötig kopiert
- Pointer werden gesendet, obwohl das Objekt danach ungültig wird
- Producer erzeugt schneller Daten als Consumer sie verarbeitet



Situation	Mutex	Queue
Gemeinsame Variable schützen	geeignet	eher nicht
Daten von Task A nach Task B übertragen	eher nicht	geeignet
Zugriff auf Display, I2C oder Serial schützen	geeignet	indirekt möglich
Messwerte puffern	eher nicht	geeignet
Zeitliche Entkopplung von Tasks	nein	ja

Merksatz

Mutex: schützt eine gemeinsam genutzte Ressource.

Queue: überträgt Nachrichten zwischen Tasks.



- 1 Rückblick
- 2 Superloop
- 3 `yield()`, `delay()` und Watchdog
- 4 Kooperatives Scheduling
- 5 RTOS
- 6 Dual-Core und echte parallele Aufgaben
- 7 Datenblatt-Aufgabe**
- 8 EEPROM
- 9 Sleep Modes



Aufgabe

Nutzen Sie das Datenblatt des AT24C256 und bestimmen Sie:

- 1 die 7-bit I²C-Adresse in Hex, wenn A0 = GND und A1 = GND
- 2 die drei weiteren möglichen 7-bit I²C-Adressen
- 3 die maximale SCL-Frequenz bei Betrieb mit 3,3 V

Hinweis

Gesucht sind 7-bit-Adressen, nicht das 8-bit-Adressbyte mit R/W-Bit.



Datenblatt AT24C128 /
AT24C256

[https://ww1.microchip.com/
downloads/en/DeviceDoc/
doc0670.pdf](https://ww1.microchip.com/downloads/en/DeviceDoc/doc0670.pdf)



- ① Rückblick
- ② Superloop
- ③ `yield()`, `delay()` und Watchdog
- ④ Kooperatives Scheduling
- ⑤ RTOS
- ⑥ Dual-Core und echte parallele Aufgaben
- ⑦ Datenblatt-Aufgabe
- ⑧ EEPROM**
- ⑨ Sleep Modes



- EEPROM ist nichtflüchtiger Speicher
- Daten bleiben nach Reset oder Ausschalten erhalten
- Geeignet für kleine Konfigurationsdaten

Typische Anwendungen

- Geräteeinstellungen
- Kalibrierwerte
- Zählerstände
- zuletzt gewählter Betriebsmodus

Nicht geeignet

EEPROM ist nicht für häufige Messdaten oder kontinuierliches Logging geeignet.



- Der ESP8266 besitzt kein klassisches externes EEPROM
- Die Arduino-EEPROM-Schnittstelle wird über Flash-Speicher emuliert
- Vor der Nutzung muss ein Speicherbereich reserviert werden
- Änderungen werden erst mit `EEPROM.commit()` dauerhaft gespeichert

Wichtige Funktionen

Funktion	Bedeutung
<code>EEPROM.begin(size)</code>	EEPROM-Emulation starten
<code>EEPROM.read(address)</code>	Byte lesen
<code>EEPROM.write(address, value)</code>	Byte schreiben
<code>EEPROM.commit()</code>	Änderungen in Flash speichern
<code>EEPROM.end()</code>	Speicher freigeben, vorher commit



```
#include <EEPROM.h>

const int eepromSize = 16;
const int address = 0;

void setup() {
    Serial.begin(115200);

    EEPROM.begin(eepromSize);

    // Write one byte
    EEPROM.write(address, 42);

    // Save changes to flash
    EEPROM.commit();

    // Read one byte
    byte value = EEPROM.read(address);

    Serial.print("Value: ");
    Serial.println(value);
}

void loop() {
}
```

- Eine Adresse speichert ein Byte
- Wertebereich: 0 bis 255
- commit() ist beim ESP8266 notwendig

Merksatz

write() ändert nur die RAM-Kopie.
commit() schreibt dauerhaft in den Flash.



- EEPROM speichert nur einzelne Bytes
- Die Bedeutung entsteht erst durch das Programm
- Ein Byte kann z. B. Zahl, Modus, Zeichen oder Teil einer Struktur sein

Beispiel

Eine Konfiguration besteht aus mehreren zusammengehörigen Bytes:

Modus	Intervall	Kalibrierwert
-------	-----------	---------------

Problem

Beim Start liest das Programm Bytes aus dem EEPROM.

Es muss prüfen können, ob diese Bytes wirklich eine gültige Konfiguration bilden.



- Eine Magic Number ist eine feste Kennung am Anfang der Struktur
- Beim Lesen wird geprüft, ob diese Kennung vorhanden ist
- Fehlt die Kennung, wird eine Standardkonfiguration erzeugt
- Zusätzlich können Version und Strukturgröße gespeichert werden

Prinzip

```
magic == CONFIG_MAGIC
```

Nutzen

- Erkennung eines uninitialisierten EEPROM-Bereichs
- Erkennung alter oder inkompatibler Daten
- kontrollierter Start mit Standardwerten



```
#include <EEPROM.h>

const int eepromSize = 64;
const int address = 0;

const uint32_t CONFIG_MAGIC = 0x43464731;
const uint16_t CONFIG_VERSION = 1;

struct Config {
    uint32_t magic;
    uint16_t version;
    uint16_t size;
    uint32_t bootCounter;
    float calibration;
};

Config config;

void setDefaultConfig() {
    config.magic = CONFIG_MAGIC;
    config.version = CONFIG_VERSION;
    config.size = sizeof(Config);
    config.bootCounter = 0;
    config.calibration = 1.23;
}
```

- magic erkennt gültige Daten
- version erkennt alte Formate
- size erkennt Layoutänderungen

Hinweis

0x43464731 entspricht der Kennung CFG1.



```
bool isConfigValid(const Config& c) {  
    return c.magic == CONFIG_MAGIC &&  
           c.version == CONFIG_VERSION &&  
           c.size == sizeof(Config);  
}  
  
void setup() {  
    Serial.begin(115200);  
    EEPROM.begin(eepromSize);  
  
    EEPROM.get(address, config);  
  
    if (!isConfigValid(config)) {  
        setDefaultConfig();  
    }  
  
    config.bootCounter++;  
  
    EEPROM.put(address, config);  
    EEPROM.commit();  
  
    Serial.print("Boots: ");  
    Serial.println(config.bootCounter);  
  
    Serial.print("Calibration: ");  
    Serial.println(config.calibration);  
}
```

- Erst lesen
- Dann Gültigkeit prüfen
- Bei Fehler Standardwerte setzen
- Danach veränderte Struktur speichern

Merksatz

EEPROM-Daten niemals blind als gültige Konfiguration behandeln.



- Flash-Speicher hat begrenzte Schreibzyklen
- Nicht bei jeder `loop()`-Ausführung schreiben
- Nur speichern, wenn sich ein Wert wirklich geändert hat
- Schreibzugriffe bündeln

Gute Praxis

- Konfiguration selten speichern
- Messdaten nicht im EEPROM loggen
- Vor dem Schreiben prüfen, ob sich Daten geändert haben

Fehlerbild

Häufiges `commit()` kann den Flash unnötig stark belasten.

EEPROM: Schreiben nur bei Änderung



```
#include <EEPROM.h>
const int address = 0;

void saveMode(byte newMode) {
    byte oldMode = EEPROM.read(address);

    // Write only if value changed
    if (oldMode != newMode) {
        EEPROM.write(address, newMode);
        EEPROM.commit();

        Serial.println("Mode saved");
    }
}

void setup() {
    Serial.begin(115200);
    EEPROM.begin(16);

    saveMode(3);
}
```

- Reduziert Schreibzugriffe
- Schont Flash-Speicher
- Sinnvoll für Konfigurationswerte

Regel

Lesen ist unkritisch.
Schreiben sollte bewusst erfolgen.



- EEPROM speichert kleine Daten dauerhaft
- Beim ESP8266 wird EEPROM im Flash emuliert
- `EEPROM.begin(size)` initialisiert den Speicherbereich
- `EEPROM.commit()` macht Änderungen dauerhaft
- Häufiges Schreiben vermeiden

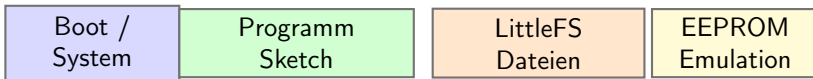
Merksatz

EEPROM ist für Konfigurationen gedacht, nicht für kontinuierliche Datenaufzeichnung.

Flash-Speicher: ein Speicher, mehrere Bereiche



- Der ESP speichert Programm und Daten im Flash-Speicher
- Der Flash wird durch das Board-Layout logisch aufgeteilt
- LittleFS und EEPROM-Emulation nutzen reservierte Flash-Bereiche



Ein physischer Flash-Speicher wird in mehrere logische Bereiche aufgeteilt.

Einordnung

- **Programm:** enthält den kompilierten Code
- **LittleFS:** speichert Dateien mit Namen und Inhalt
- **EEPROM-Emulation:** speichert kleine Werte über die EEPROM-Bibliothek



- Mikrocontroller nutzen verschiedene Speicherarten für unterschiedliche Aufgaben
- Entscheidend sind Flüchtigkeit, Schreibverhalten und typische Nutzung
- Beim ESP8266/ESP32 wird EEPROM meist im Flash-Speicher emuliert

Speicher	Verhalten	Typische Daten	Verwendung
RAM	flüchtig, schnell	Variablen, Stack, Heap	laufende Programmdateien
EEPROM	nichtflüchtig, kleine Daten	Modus, Kalibrierwert, Zähler	selten geänderte Konfiguration
Flash	nichtflüchtig, blockweise beschreibbar	Firmware, LittleFS, Webseiten	Programm und Dateisystem

Einordnung

Beim ESP8266/ESP32 ist EEPROM meist kein eigener Speicherbaustein.
Die EEPROM-Bibliothek speichert die Daten in einem reservierten Flash-Bereich.



```
Serial.println("Text");  
  
Serial.println(F("Text"));  
  
String texts[30];  
  
for (int i = 0; i < 30; i++) {  
    texts[i] =  
        "Langer dynamischer Text im RAM";  
}
```

Messung

Zustand	Freier Heap
vor String	334008 Byte
nach String	329088 Byte
Differenz	4920 Byte

- `ESP.getFreeHeap()` misst dynamischen Speicher
- String-Objekte belegen Heap
- `F("Text")` vermeidet unnötige RAM-Nutzung bei konstantem Text



Speicherart	Geeignet für	Typische Daten
EEPROM	kleine Werte	Modus, Zähler, Kalibrierwert
LittleFS	Dateien	Konfiguration, Textdateien, Webseiten

Einordnung

EEPROM speichert einzelne Werte oder kleine Strukturen.

LittleFS speichert Dateien im Flash-Speicher.



Was ist LittleFS?

LittleFS ist ein kleines Dateisystem im Flash-Speicher des ESP.

- Dateien werden über Namen angesprochen
- Inhalte können Text, JSON, HTML oder CSS sein
- Geeignet für Daten, die selten geändert und oft gelesen werden

Datei	Typischer Inhalt
/config.txt	Einstellungen
/config.json	strukturierte Konfiguration
/index.html	Webseite für Webserver
/log.txt	kleine Logdatei



```
#include <LittleFS.h>

void setup() {
  Serial.begin(115200);

  if (!LittleFS.begin()) {
    Serial.println("LittleFS mount failed");
    return;
  }

  Serial.println("LittleFS ready");
}

void loop() {
}
```

- LittleFS.begin() bindet das Dateisystem ein
- Ohne erfolgreiche Initialisierung kein Dateizugriff



```
#include <LittleFS.h>

void setup() {
    Serial.begin(115200);

    LittleFS.begin();

    File file = LittleFS.open("/config.txt", "w");

    if (!file) {
        Serial.println("Open failed");
        return;
    }

    // Write configuration data
    file.println("mode=3");
    file.println("interval=1000");

    file.close();

    Serial.println("File written");
}

void loop() {
}
```

Was passiert?

- LittleFS.begin() bindet das Dateisystem ein
- open(..., "w") öffnet die Datei zum Schreiben
- Existiert die Datei nicht, wird sie erstellt
- Existiert sie bereits, wird sie überschrieben
- println() schreibt Textzeilen in die Datei
- close() schließt die Datei sauber



```
#include <LittleFS.h>

void setup() {
    Serial.begin(115200);

    LittleFS.begin();

    File file = LittleFS.open("/config.txt", "r");

    if (!file) {
        Serial.println("Open failed");
        return;
    }

    // Read file content
    while (file.available()) {
        Serial.write(file.read());
    }

    file.close();
}

void loop() {
}
```

Was passiert?

- LittleFS.begin() bindet das Dateisystem ein
- open(..., "r") öffnet die Datei zum Lesen
- Falls die Datei fehlt, schlägt das Öffnen fehl
- available() prüft, ob noch Daten vorhanden sind
- read() liest jeweils ein Byte
- Serial.write() gibt dieses Byte auf der seriellen Schnittstelle aus
- close() schließt die Datei



- Der ESP8266 nutzt externen Flash-Speicher
- Der Flash wird logisch in mehrere Bereiche aufgeteilt
- Ein Bereich enthält das Programm
- Ein anderer Bereich kann ein Dateisystem enthalten

Einordnung

- **SPIFFS**: älteres Flash-Dateisystem
- **LittleFS**: heutige Alternative mit besserer Verzeichnisunterstützung
- Beide speichern Dateien im Flash-Speicher

Wichtig

Die Größe des Dateisystems hängt vom gewählten Flash-Layout ab.
In der Arduino-IDE wird das unter Werkzeuge bzw. über die Board-Optionen eingestellt.



- 1 Rückblick
- 2 Superloop
- 3 `yield()`, `delay()` und Watchdog
- 4 Kooperatives Scheduling
- 5 RTOS
- 6 Dual-Core und echte parallele Aufgaben
- 7 Datenblatt-Aufgabe
- 8 EEPROM
- 9 Sleep Modes**

Warum Sleep Modes?



Anforderung

- Sensorwerte erfassen
- Daten übertragen
- auf Ereignisse reagieren
- möglichst lange mit Akku laufen

Konflikt

- aktive CPU verbraucht Energie
- Funkmodul verbraucht viel Energie
- Peripherie verbraucht auch im Leerlauf Strom
- dauerhafte Aktivität verkürzt die Laufzeit

Kernaussage

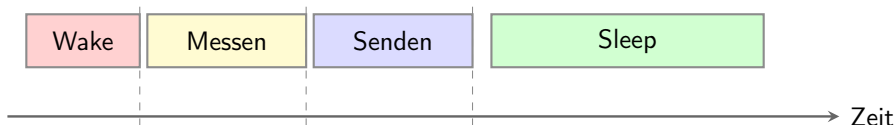
Ein IoT-Gerät muss nicht ständig aktiv sein.

Es muss nur dann aktiv sein, wenn eine Aufgabe ansteht.

Entwurfsfrage

Welche Teile des Systems müssen wirklich wach bleiben?

Grundprinzip: Aktivität statt Dauerbetrieb



Aktive Zeit kurz halten → Schlafzeit maximieren

- Energie wird über Zeit verbraucht
- Kurze aktive Phasen reduzieren den mittleren Strom
- Sleep Modes sind daher ein Systemkonzept, kein einzelner Befehl



Modus	CPU	Funkmodul	Typischer Einsatz	ESP8266 Strom
Aktiv	an	an	Rechnen, Messen, Kommunikation	ca. 80 mA
Modem Sleep	an	reduziert/aus	CPU arbeitet, WLAN nicht dauerhaft aktiv	ca. 15 mA
Light Sleep	pausiert	reduziert/aus	kurze Pausen, schneller Wake-up	ca. 0,9 mA
Deep Sleep	aus	aus	lange Pausen, Batteriebetrieb	ca. 20 μ A
Power Off	aus	aus	externe Abschaltung	-

Wichtige Unterscheidung

Je tiefer der Schlafmodus, desto geringer der Stromverbrauch.

Dafür dauert das Aufwachen länger und mehr Zustand geht verloren.

Entwurfsfrage: Was muss wach bleiben?



- Muss die CPU weiterlaufen?
- Muss WLAN verbunden bleiben?
- Muss ein GPIO das System wecken?
- Muss Zustand gespeichert werden?
- Wie schnell muss das System reagieren?

Beispiele

- Wetterstation: Deep Sleep
- Bediengerät: Light Sleep
- WLAN-Gerät mit kurzen Pausen: Modem Sleep
- Extern geschalteter Sensor: Power Off



Mögliche Modi

```
#include <ESP8266WiFi.h>

void setup() {
  Serial.begin(115200);
  delay(5000);

  WiFi.mode(WIFI_STA);

  // Select WiFi sleep behavior
  WiFi.setSleepMode(WIFI_MODEM_SLEEP);

  Serial.println("WiFi sleep configured");
}

void loop() {
  // Sketch continues to run
  delay(1000);
}
```

Modus	Bedeutung
WIFI_NONE_SLEEP	WLAN bleibt dauerhaft aktiv
WIFI_MODEM_SLEEP	Funkmodul schläft zwischen WLAN-Aktivität
WIFI_LIGHT_SLEEP	Funkmodul und CPU können in Pausen schlafen

Einordnung

WiFi.setSleepMode(...) betrifft nur das WLAN-Verhalten im laufenden Betrieb.

Es ist kein ESP.deepSleep(...).



```
#include <ESP8266WiFi.h>

void setup() {
    Serial.begin(115200);

    // Turn WiFi off
    WiFi.mode(WIFI_OFF);
    WiFi.forceSleepBegin();
    delay(1);

    Serial.println("WiFi is off");
}

void loop() {
    // Work without WiFi
    Serial.println("CPU still active");
    delay(1000);
}
```

- WLAN ist ein relevanter Verbraucher
- Abschalten sinnvoll bei lokalen Aufgaben
- CPU kann trotzdem weiterarbeiten

Praxis

Nicht jedes Projekt benötigt WLAN dauerhaft.



```
#include <ESP8266WiFi.h>

void setup() {
  Serial.begin(115200);
  delay(5000);

  WiFi.mode(WIFI_STA);

  // Allow WiFi modem and CPU
  // to sleep during idle phases
  WiFi.setSleepMode(WIFI_LIGHT_SLEEP);

  Serial.println("Light sleep configured");
}

void loop() {
  // During delay(), the ESP8266
  // can enter light sleep
  delay(1000);

  Serial.println("awake");
}
```

Was passiert?

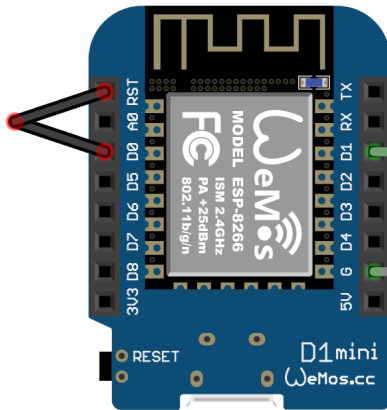
- Der Sketch läuft grundsätzlich weiter
- In Leerlaufphasen darf die CPU pausieren
- Das WLAN-Funkmodul darf ebenfalls schlafen
- RAM und Programmzustand bleiben erhalten
- Nach dem Aufwachen läuft der Code weiter

Deep Sleep: Ablauf nach dem Aufwachen



Nach Deep Sleep läuft das Programm wieder von vorn.
Variablen im RAM sind nicht mehr gültig.

- Deep Sleep ist kein normales Warten
- Der Wake-up verhält sich wie ein Neustart
- Zustand muss bei Bedarf extern oder im RTC-Speicher abgelegt werden



Timer-Wakeup aus Deep Sleep

- D0 entspricht GPIO16
- GPIO16 muss mit RST verbunden werden
- Beim Aufwachen zieht GPIO16 den Reset-Pin kurz auf Low
- Dadurch startet der ESP8266 neu

Beschaltung

D0 / GPIO16 → RST

kein zusätzlicher Widerstand nötig



```
void setup() {  
  Serial.begin(115200);  
  delay(5000);  
  
  Serial.println("Wake up");  
  
  // Do useful work here  
  delay(500);  
  
  Serial.println("Go to deep sleep");  
  
  // Sleep for 10 seconds  
  ESP.deepSleep(10e6);  
}  
  
void loop() {  
  // Not reached after deepSleep()  
}
```

- CPU wird abgeschaltet
- Programm startet nach Wake-up neu
- loop() wird hier nicht dauerhaft verwendet

Wichtig

Beim ESP8266 muss für Timer-Wake-up typischerweise GPIO16 mit RST verbunden werden.



```
const int sensorPin = A0;
const int threshold = 700;

void setup() {
  Serial.begin(115200);
  delay(5000);

  int value = analogRead(sensorPin);

  Serial.print("Sensor: ");
  Serial.println(value);

  if (value > threshold) {
    // WiFi only if communication is needed
    WiFi.mode(WIFI_STA);
    WiFi.begin(ssid, password);

    while (WiFi.status() != WL_CONNECTED) {
      delay(100);
    }

    Serial.println("Send data here");
  }

  // Wake up with WiFi disabled
  ESP.deepSleep(30e6, WAKE_RF_DISABLED);
}
```

Prinzip

- ESP wacht zyklisch auf
- WLAN bleibt zunächst aus
- Sensor wird gemessen
- Nur bei Grenzwertüberschreitung wird WLAN gestartet
- Danach geht der ESP wieder in Deep Sleep

Nutzen

WLAN ist nur aktiv, wenn wirklich Daten übertragen werden müssen.



Speicher	Nach Reset	Nach Deep Sleep	Typische Nutzung
RAM	gelöscht	gelöscht	laufende Variablen
RTC-Speicher	meist gelöscht	teilweise erhalten	Wake-Zähler, kurzer Zustand
EEPROM / Flash	erhalten	erhalten	Konfiguration, Kalibrierung

Merksatz

Nicht jeder Zustand muss dauerhaft gespeichert werden.



```
uint32_t wakeCounter = 0;

void setup() {
  Serial.begin(115200);
  delay(5000);
  // Read counter from RTC memory
  ESP.rtcUserMemoryRead(
    0, // start address in RTC memory
    &wakeCounter, // destination variable
    sizeof(wakeCounter)
  );

  wakeCounter++;

  Serial.print("Wake counter: ");
  Serial.println(wakeCounter);

  // Store counter before deep sleep
  ESP.rtcUserMemoryWrite(
    0, // start address in RTC memory
    &wakeCounter, // destination variable
    sizeof(wakeCounter)
  );

  // Sleep for 10 seconds
  ESP.deepSleep(10e6);
}
```

Was passiert?

- Der ESP wacht auf
- setup() startet neu
- Zähler wird aus RTC-Speicher gelesen
- Zähler wird um eins erhöht
- Zähler wird wieder gespeichert
- ESP geht erneut schlafen

Kernaussage

RTC-Speicher kann kleine Werte über Deep Sleep hinweg behalten.



- Normale Variablen gehen im Deep Sleep verloren
- Auch GPIOs werden nach dem Aufwachen neu initialisiert
- Der ESP32 kann bestimmte GPIO-Pegel im Deep Sleep halten
- Dafür wird GPIO-Hold verwendet

Prinzip

Pin setzen → GPIO-Hold aktivieren → Deep Sleep starten

Wichtig

Das Halten eines GPIO-Pegels ist kein Speichern von Programmdateien.

Es hält nur den elektrischen Zustand eines Pins.



```
#include "driver/gpio.h"
#include "esp_sleep.h"
const gpio_num_t holdPin = GPIO_NUM_25;
void setup() {
    Serial.begin(115200);
    delay(5000);

    // Release possible hold from previous sleep cycle
    gpio_deep_sleep_hold_dis();
    gpio_hold_dis(holdPin);
    pinMode(25, OUTPUT);

    // Set output state before deep sleep
    digitalWrite(25, HIGH);

    // Hold this pin state
    gpio_hold_en(holdPin);

    // Keep GPIO hold active during deep sleep
    gpio_deep_sleep_hold_en();

    // Wake up after 10 seconds
    esp_sleep_enable_timer_wakeup(10ULL * 1000000ULL);

    // Start deep sleep
    esp_deep_sleep_start();
}
```

Was passiert?

- GPIO25 wird als Ausgang gesetzt
- der Pin wird auf HIGH gelegt
- `gpio_hold_en()` fixiert diesen Zustand
- `gpio_deep_sleep_hold_en()` hält ihn auch im Deep Sleep
- nach 10 Sekunden wacht der ESP32 wieder auf

Beispiel

Eine LED an GPIO25 kann während Deep Sleep eingeschaltet bleiben.



Modus	Aktiv	Anwendung
Modem Sleep	CPU läuft, WLAN reduziert	Programm läuft weiter, Funk wird gespart
Light Sleep	RAM bleibt, CPU pausiert	kurze Pausen, schnelle Rückkehr
Deep Sleep	fast alles aus	lange Messintervalle, Neustart nach Wake-up
Power Off	ESP stromlos	externe Schaltung schaltet wieder ein

Merksatz

Tieferer Schlaf spart mehr Energie, verliert aber den Zustand.